



university of
 groningen

Potential-Based Reward Shaping for Planar Pushing

Single-Agent Control vs. Asymmetric Self-Play

Vlad Iftime s3426394

Faculty of Science and Engineering, Artificial Intelligence
University of Groningen

June 2026

1. Introduction & Research Questions

Outline



1. **Introduction & Research Questions**

2. **Background & Related Work**

reward shaping · curriculum & self-play · push mechanics & SE(2)

3. **Problem Definition & MDP**

4. **Implementation**

five models: PPO-Baseline, PPO-PBRS, PPO-Curriculum, ASP-dPose, TASP-dPose

5. **Evaluation**

30 held-out scenes · 20 trials · Isaac vs gym

6. **Results**

7. **Discussion**

8. **Conclusion & Takeaways**

Research Questions



Three Research Questions

1. **RQ1:** Does asymmetric self-play match single-agent training in a contact-rich planar goal-reaching task under a single-GPU budget?
2. **RQ2:** Which single dimension, among goal coupling, reward formulation, imitation signal, goal representation, and compute scale, accounts for the self-play outcome?
3. **RQ3:** What mechanism prevents self-play from generalising its training-time success?

2. Background & Related Work



Reward Shaping (PBRs)

Potential-based reward shaping :

$$\mathcal{R}'(s, a, s') = \mathcal{R}(s, a, s') + \underbrace{\gamma\Phi(s') - \Phi(s)}_F$$

- **Policy invariance:** $Q'(s, a) = Q(s, a) - \Phi(s)$ – the optimal policy is unchanged for *any* state potential Φ .
- **Episodic validity :** $\gamma_{\text{shaping}}=1$ is safe – $\sum F = \Phi(s_T) - \Phi(s_0)$ is bounded.
- **Dynamic / arbitrary potentials** – basis for our exponential Φ .

Why it matters here: lets us design a dense, well-behaved reward *without* changing the task's optimum – the foundation of every model in this thesis.



Curriculum & Asymmetric Self-Play

Forced curricula

- **Surveys**
- **Reverse / goal** : start near the goal, expand outward
- **Precision** : gradually tighten the success tolerance

Our use: a forced 3-phase pos→rot curriculum (PPO-Curriculum)

Automatic (self-play)

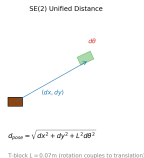
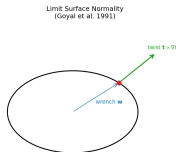
- **ASP** : Alice/Bob adversarial game + ABC
- **Time-based Alice** : $R_A \propto t_B - t_A$, self-regulating
- **Regret-based**

Our use: ASP-dPose (outcome) and TASP-dPose (time-based)

Geometry & Mechanics of Planar Pushing

Push mechanics

- **Quasi-static pushing:** motion set by the friction limit surface.
- **Limit-surface normality:** every push couples translation and rotation.



SE(2) geometry

- **Unified distance:** a single scalar that combines position and orientation error, weighted by the object's physical size L
- **Equivariant manipulation:** exploiting SE(2)/SO(2) symmetry.

Our use: the SE(2) d_{pose} provides a unified reward

3. Problem Definition & MDP

Planar-Pushing MDP – State, Action & Transition



State $s \in \mathbb{R}^{28}$: EE pose (6) | object pose (14) | goal pose (6) | errors (2)

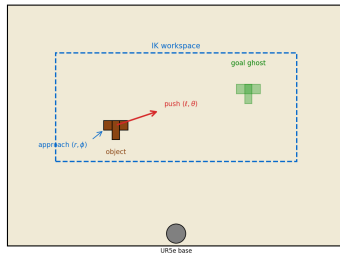
Action $a \in \mathbb{R}^4$ (object-relative): (r, ϕ, ℓ, θ) – 21 bins/dim, 194 481 discrete actions

Transition : $t \propto \nabla f(w)$ – limit-surface normality couples **every** push with rotation

Success: pos < 5 cm **and** rot < 0.2 rad.

Validation: 30 scenes $\times$ 20 trials $\times$ up to 30 pushes

Planar Pushing Task — Object-Relative Action (r, ϕ, ℓ, θ)





How the Fractional Reward Failed

Fractional improvement formula:

$$R = \underbrace{\alpha \cdot \frac{d_{\text{prev}} - d_{\text{now}}}{d_{\text{prev}}}}_{\text{improvement}} - \underbrace{\beta_{\text{pos}} \cdot d_{\text{now}}}_{\text{pos penalty}} - \underbrace{\beta_{\text{rot}} \cdot y_{\text{now}}}_{\text{rot penalty}}, \quad \alpha=3.0, \beta_{\text{pos}}=0.5, \beta_{\text{rot}}=0.25$$

Worked example – pushing toward a goal 30 cm away, 1 cm improvement:

$$R = 3.0 \cdot 0.01 / 0.30 - 0.5 \cdot 0.29 - 0.25 \cdot 0.5 = -0.17$$

Problem 1 – Negative reward for progress: the penalty terms dominate the improvement.

Problem 2 – Near-zero variance: 99% of pushes produce $R \in [-0.5, +0.1]$.



How PBRS Fixes the Reward

PBRS potential: $\Phi(s) = w \cdot e^{-k \cdot d^2}$, $k=30$, $w=10$

Same push, same 1 cm improvement:

$$R = w \cdot [e^{-k \cdot d_{\text{now}}^2} - e^{-k \cdot d_{\text{prev}}^2}] = 10 \cdot [e^{-30 \cdot 0.29^2} - e^{-30 \cdot 0.30^2}] = +0.133$$

Why this matters:

- **Correctly signed** – every push toward the goal gets positive reward, away gets negative
- **Bounded** – $\Phi(s) \in [0, w]$, so $|R| \leq w$ regardless of distance or physics glitches
- **Markovian** – depends only on current state s , not on history or d_{prev}
- **Policy-invariant** – the optimal policy under PBRS is identical to the original task

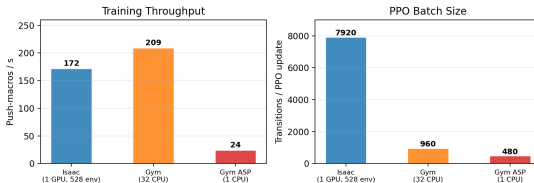
4. Implementation



Why Isaac Lab?

1. **GPU-batched parallelism** – 528 simultaneous environments produce 7,920-sample PPO batches. gym-pusht peaks at 960 – $8\times$ smaller. Large batches stabilize the non-stationary self-play gradient.
2. **No extra per-iteration cost** – self-play and single-agent run at identical wall-clock (~ 0.022 it/s). The two-agent machinery adds negligible overhead.

Isaac vs Gym — Why GPU-Batched Parallelism Matters



Throughput is comparable; batch size is the differentiator.

Why Isaac Lab? – Training demos





Results – Single-Agent Baselines vs Self-Play

Same PPO+LSTM, same push primitive, same 528-env budget, 5 seeds:

Model	Scene SR	Pos-only	Pos+rot
PPO-PBRS	79.3%	100%	69%
PPO-Curriculum	80.0%	100%	70%
ASP-dPose (outcome)	0.0%	0%	0%
TASP-dPose (time)	0.0%	0%	0%
ASP-disc (position-only)	7.3%	–	–

Single-agent baselines solve ~4 in 5 scenes; self-play transfers none. Self-play's training-time s
(5 seeds; ASP-disc validated on its own position-only disc scenes.)

All Models at a Glance



Model	Idea	Reward / curriculum
PPO-Baseline	single agent, original reward	fractional improvement (non-PBRS)
PPO-PBRS	single agent, no curriculum	two-potential PBRS (pos, rot)
PPO-Curriculum	single agent + staged curriculum	PBRS + forced pos→rot
ASP-dPose	self-play (Alice/Bob)	SE(2) PBRS, outcome Alice (+5/ - 1)
TASP-dPose	self-play (Alice/Bob)	SE(2) PBRS, time-based Alice

All five models share the same PPO+LSTM backbone and object-relative push primitive.

[Schulman et al. 2017] [Ng et al. 1999] [Sukhbaatar et al. 2018]

PPO-PBRS – Single-Agent (Recommended Baseline)



Architecture: Single PPO agent, LSTM(28→256) + MLP[512, 256] trunk, MultiCategorical head (4 dims $\times$ 21 bins). No curriculum, no Alice/Bob, no GoalEncoder.

Training: 528 envs, 15 pushes/rollout, ~ 3000 iterations ~ 1.6 days wall-clock (~ 0.022 it/s).

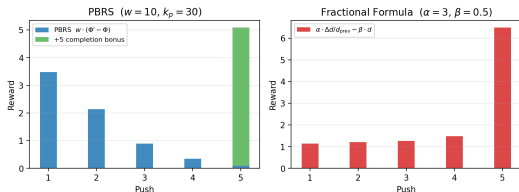
Why it works:

1. **Dense, bounded gradient** – PBRS fires on every push
2. **Fixed goal distribution** – stationary random goals; no adversarial shift
3. **Single network** – one agent, one optimizer; no multi-agent instability



PBRS vs Fractional– Reward Signal Analysis

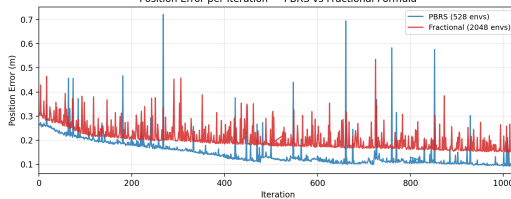
Simulated 5-Push Episode — PBRS vs Fractional Formula



Per-push reward – Fractional vs PBRS: same 5-push episode. Fractional formula: negative or zero for 80% of pushes. PBRS: correctly-signed, bounded on every push.

Dense signal on every push, bounded rewards, stable value – PBRS avoids the gradient starvation and critic instability of the old fractional formula.

Position Error per Iteration — PBRS vs Fractional Formula



Position error per iteration (TensorBoard): PBRS at 528 envs converges to lower PosErr than fractional formula at 2,048 envs – 4× more environment-efficient.

Why PBRs for ASP? – Decoupling Advantage from Value



GAE advantage decomposes into two parts:

$$\hat{A}_t = \underbrace{\sum (\gamma\lambda)^l [\Phi(s_{t+l+1}) - \Phi(s_{t+l})]}_{\text{always correctly-signed}} + \underbrace{\sum (\gamma\lambda)^l [\gamma V(s_{t+l+1}) - V(s_{t+l})]}_{\text{biased when } V \text{ is stale}}$$

Φ -term: PBRs potential difference

$\Phi(s) = w \cdot e^{-k \cdot d^2}$ increases when object moves toward goal. Correct sign on every push regardless of $V(s)$ accuracy.

V -term: value function bootstrap

Under ASP, Alice shifts goal distribution $\rightarrow V(s)$ perpetually stale. GAE bootstraps from wrong values $\rightarrow$ biased advantage.

Outcome: PBRs lifts ASP 0% $\rightarrow$ 16.7% via the Φ -term – but only partially, because the V -term bias remains.

Single-agent: no distribution shift $\rightarrow V$ accurate $\rightarrow$ both align $\rightarrow$ 80%.

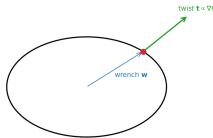
SE(2) Unified Distance – Self-Play Variants

SE(2) metric: $d_{\text{pose}} = \sqrt{dx^2 + dy^2 + L^2 d\theta^2}$

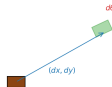
- Single scalar combining (dx, dy) and $d\theta$; L = char. length (T-block 0.07 m)
- **Single-potential PBRs:** $\Phi(s) = w e^{-k d_{\text{pose}}^2}$, $k = 30$, $w = 10$
- Eliminates competing pos/rot gradients
 - aligns reward with the coupled physics

Result: SE(2) does *not* rescue self-play – ASP-dPose reaches only 6.7% validation SR vs single-agent 80%. The self-play curriculum is the root cause, not the metric.

Limit Surface Normality
(Goyal et al. 1991)



SE(2) Unified Distance



$$d_{\text{pose}} = \sqrt{dx^2 + dy^2 + L^2 d\theta^2}$$

T-block $L = 0.07\text{m}$ (rotation couples to translation)

PPO-Curriculum – Position→Rotation Curriculum



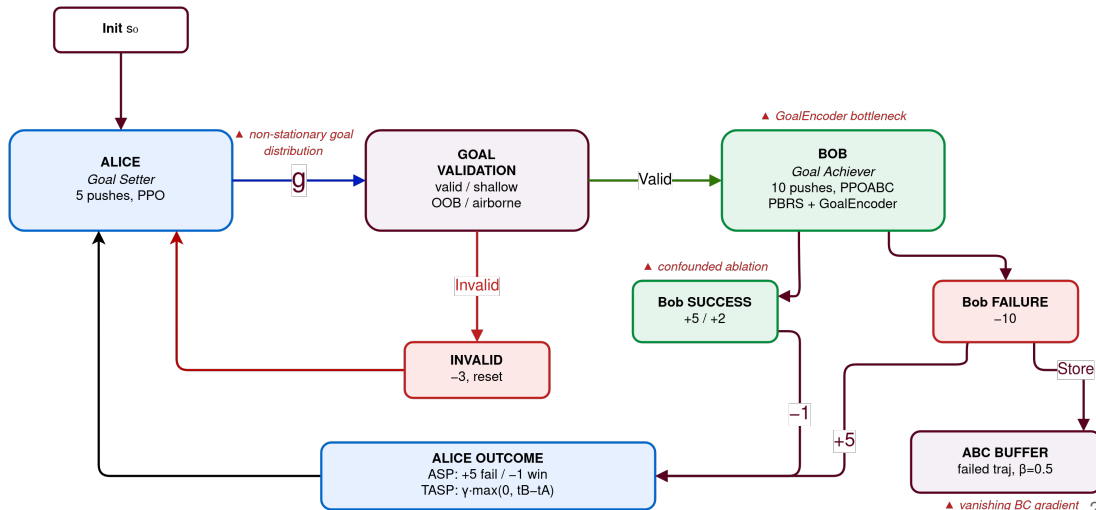
Design: Same PPO agent as PPO-PBRS + 3-phase curriculum:

1. $w_{\text{rot}} = 0$, position-only training
2. Triggered when episodic position-SR ≥ 0.5 for 20 iters: w_{rot} ramps $0 \rightarrow 10$ over 200 iters
3. Full PBRS (multi-objective)

Trigger: gates on episodic position-only SR ≥ 0.5 over a 20-iter lookback. Curriculum now activates and reaches **76.7% SR** – only 3.3pp below PPO-PBRS.



ASP Loop – Alice $\leftrightarrow$ Bob Cycle





ASP-dPose vs TASP-dPose – Alice Reward

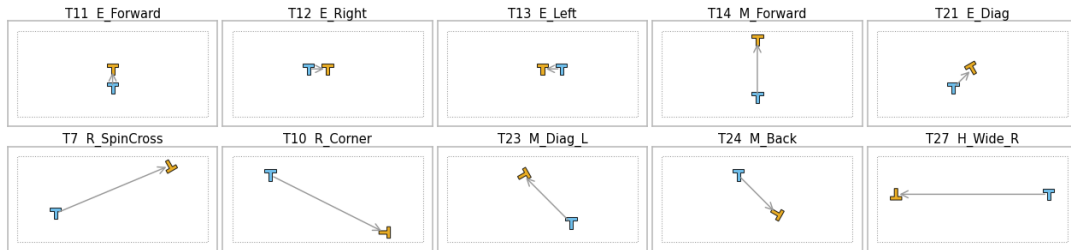
Both share the SE(2) d_{pose} Bob reward and T-block object; only Alice's reward differs.

Self-play variant	Alice reward	Valid SR
ASP-dPose (outcome-based)	$R_A = +5 \text{ fail} / -1 \text{ win}, \beta_{\text{abc}}=0.5$	6.7%
TASP-dPose (time-based)	$R_A = \gamma_{sp} \cdot \max(0, t_B - t_A), \beta_{\text{abc}}=0.5$	16.7%

Why ABC provides negligible gradient: Bob's MultiCategorical policy ($4 \text{ dim} \times 21 \text{ bins} = 194 \text{ K}$ discrete actions) concentrates probability on goal-relevant pushes. Alice's unguided actions lie outside this concentration $\rightarrow \pi/\pi_{\text{old}}$ ratio extreme $\rightarrow$ PPO clip activates $\rightarrow$ zero BC gradient despite $\beta=0.5$. Combined with non-repeatable push transitions, ABC contributes nothing.

5. Evaluation

Validation Test Suite – Top 10 Held-Out Scenes



Top row: 5 easiest scenes (3-4 of 4 models pass) | Bottom row: 5 hardest scenes (0 of 4 models pass)

Start pose Goal pose

T-block **start pose** → **goal pose**. Every model is evaluated on an identical 30-scene suite (20 trials × up to 30 pushes per scene).



What Does Success Look Like?

Position-only success



Position + Rotation success



Success gate:

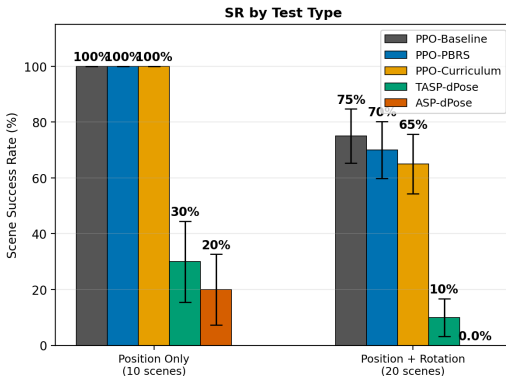
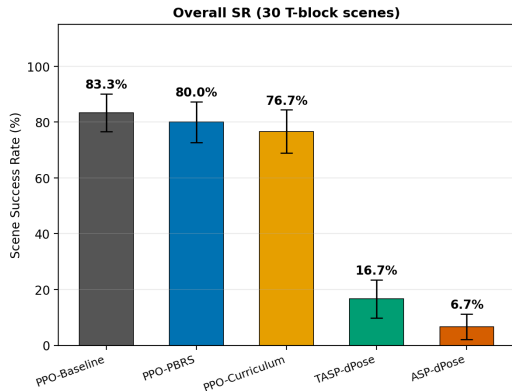
- **Object** reaches **goal**
pos < **0.05 m** **AND** **rot** < **0.2 rad**
- 10/30 scenes are pos-only (rotation ungated)
- 20/30 scenes require both
- ≥ 1 of 20 trials must pass per scene

6. Results

Multi-Model Validation



Multi-Model Validation — 30 T-block Scenes, Best-of-20, 30-Push Budget
(Error bars = ± 1 SE across scenes)





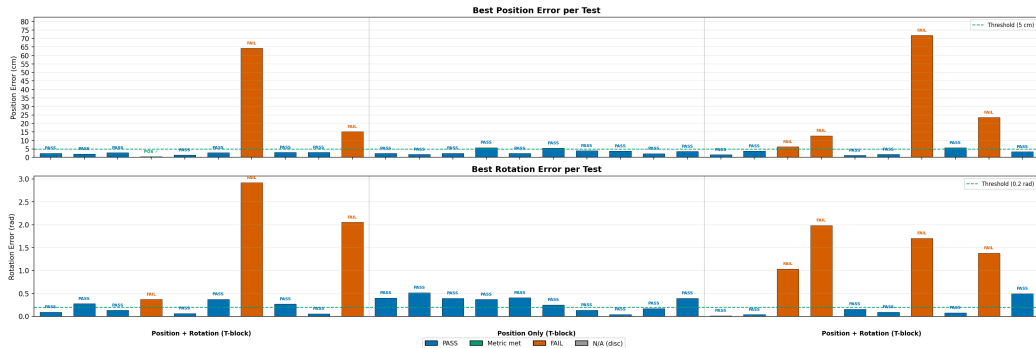
PPO-PBRS – Validation Results

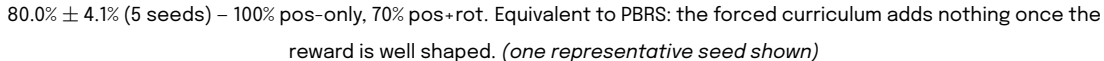
30 held-out T-block scenes (20 trials each):

- **80.0% SR** (24/30) – pos-only **100%**, pos+rot **70%**
- PosErr **0.032 m**, RotErr **0.568 rad**
(mean over all 30 scenes, incl. failures)
- Per-attempt (trial-level) SR **66%**;
scene SR counts a scene solved if any of its 20 trials passes



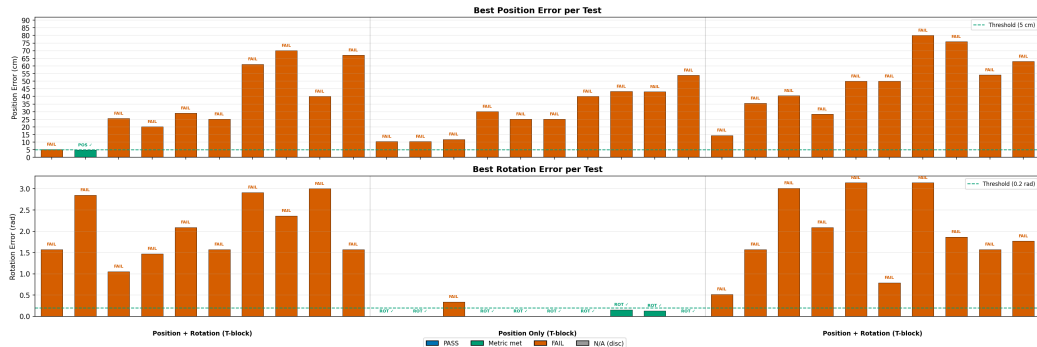
79.3% $\pm$ 3.5% (5 seeds) – 100% pos-only, 69% pos+rot. All failures are rotation-limited; position stays within tolerance on every scene. (*one representative seed shown*)







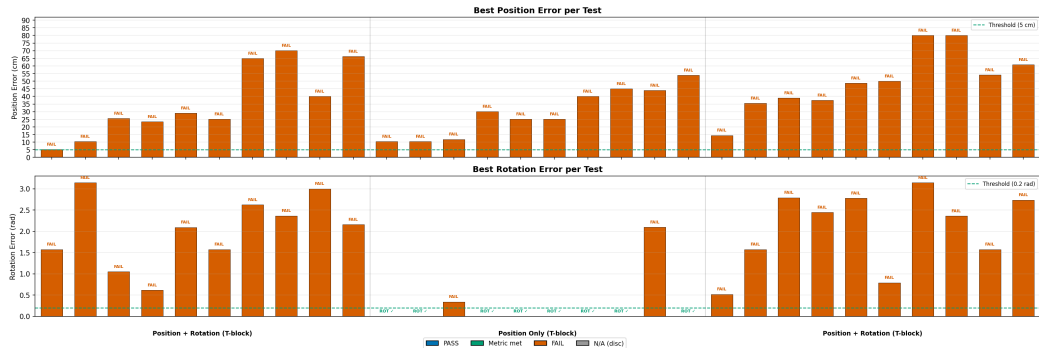
TASP-dPose – Per-Test Error



0.0% (4 seeds) – no held-out scene solved; rotation never learned (RotErr ~ 1.35 rad). Training-time success does not transfer. (*one representative seed shown*)

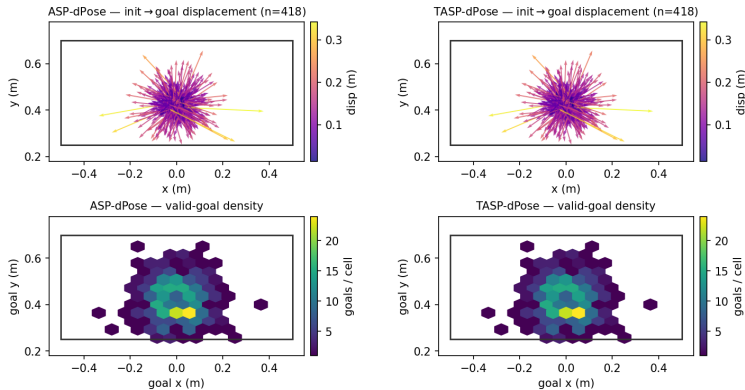


ASP-dPose – Per-Test Error



0.0% (5 seeds) – no held-out scene solved; rotation never learned (RotErr ~ 1.44 rad). Outcome-based Alice transfers nothing. (*one representative seed shown*)

ASP vs TASP training



ASP-dPose (outcome) vs TASP-dPose (time): goal displacement (top) and valid-goal density (bottom).

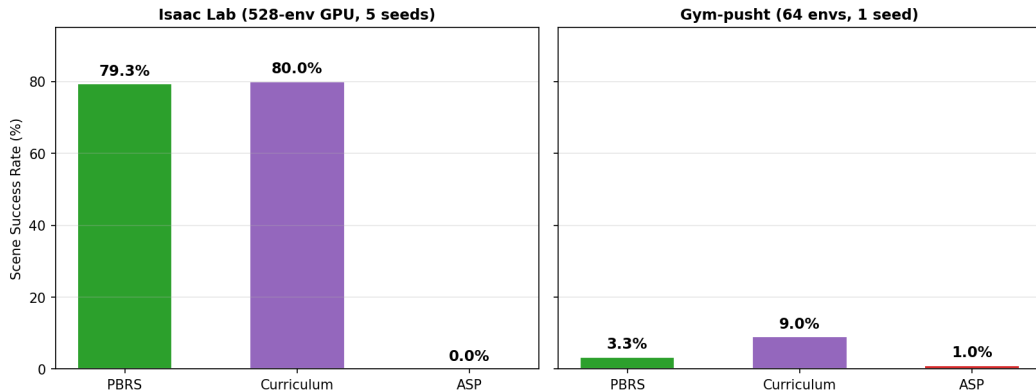
PPO-PBRS – Validation Runs (overhead)



Cross-Environment Validation – Isaac vs Gym



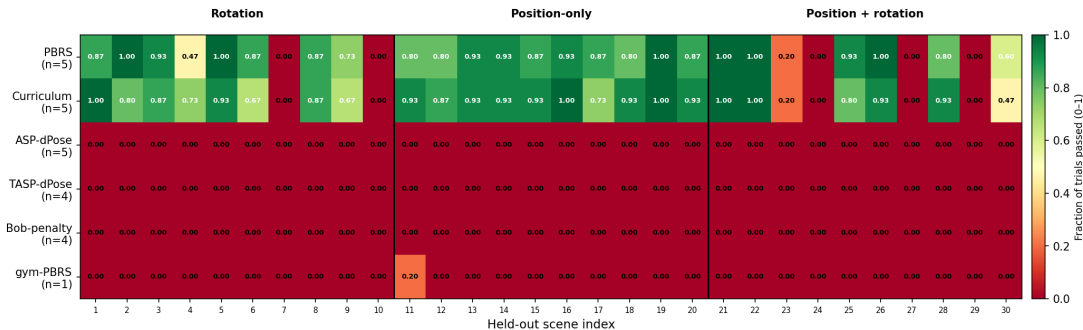
Cross-Environment Validation — Isaac Lab vs Gym-pusht (held-out scene SR)



The ordering single-agent $\approx$ curriculum $\gg$ self-play holds in both simulators.



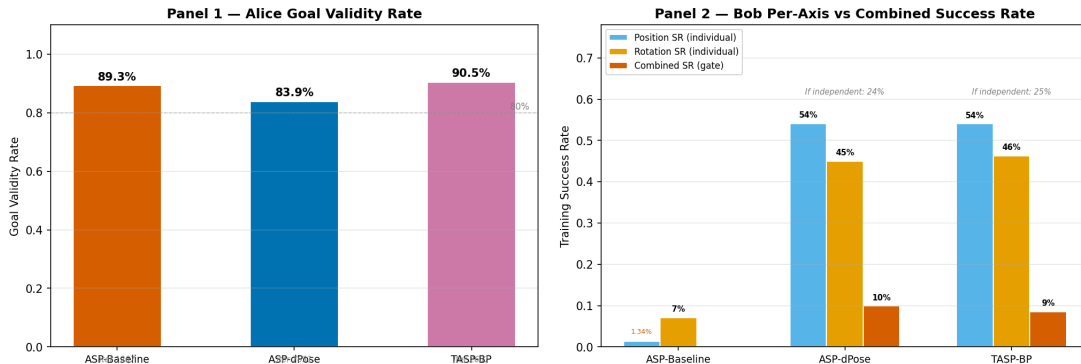
Per-Scene Outcomes – Isaac vs Gym



Colour: red (0.0) → yellow (0.5) → green (1.0). Numbers = fraction of trials passed per scene.

7. Discussion

ASP Diagnostic – Alice & Bob Training Dynamics



Left: Alice validity converges to 83–91% across all three ASP variants at different training budgets. **Right:** Bob learns position and rotation to $\sim 50\%$ each under PBRs – but the combined gate caps at $\sim 10\%$, $2.5\times$ below the independence product.



ASP – The Empirical Result

No ASP configuration exceeds 16.7% against an 80% single-agent ceiling:

ASP configuration	Valid SR	Pos+rot SR
Original fractional reward, 2,048 envs	0.0%	0%
PBRS outcome-based (ASP-dPose)	6.7%	0%
PBRS time-based (TASP-dPose)	16.7%	10%
PBRS time-based + Bob penalty	0.0%	0%
gym-pusht (different sim, same gate)	13.3%	0%
Single-agent ceiling (PPO-PBRS)	80.0%	70%

- Self-play and single-agent cost the same per iteration (~ 0.022 it/s) – the gap is **structural**, not a budget artifact

8. Conclusion & Takeaways

Conclusion – Self-Play vs Single-Agent (RQ1)



RQ1 – Self-play does not match single-agent training

Single-agent: **79–80%** held-out (5 seeds), every position-only scene solved. Self-play: **0.0%** in every seed, every reward formulation, every environment count (256–2,048). A categorical failure to generalise.

[Plappert et al. 2021] [Sukhbaatar et al. 2018]

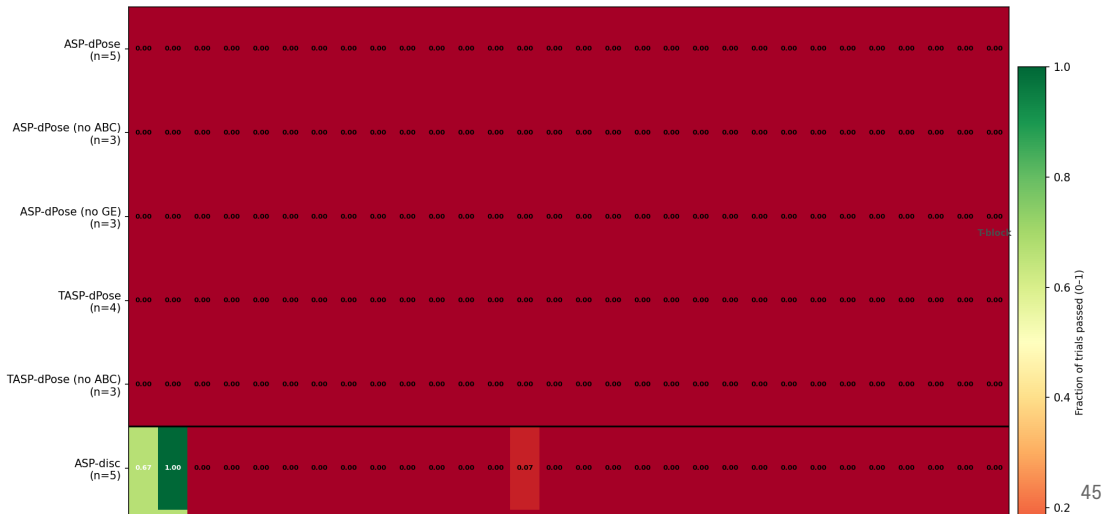
Conclusion – Which Setup Dimension (RQ2)



RQ2 – The coupled multi-objective gate is the factor

The coupled gate turns the disc's **7.3%** into **0.0%** on the T-block. Compute scale, the imitation signal, and the goal representation contribute nothing: scale flat at zero; clip fraction 0.000.

[Plappert et al. 2021] [**Sukhbaatar et al. 2018**] [Berner et al. 2019]



Conclusion – Mechanism (RQ3)



RQ3 – A generalisation failure, not a skill-composition failure

Self-play logs **77.9%** (disc) and **35.8%** (T-block) training-time success; none transfers. The value function is stale under the non-stationary goal distribution: a generalisation failure, not a skill-composition failure.

[Plappert et al. 2021] [Sukhbaatar et al. 2018] [Schulman et al. 2017]

Conclusion – Reward Efficiency (Supporting RQ)



Supporting RQ – PBRs solves the task; efficiency comparison pending

PBRs solves **79.3%** of held-out scenes (5 seeds); policy invariance (Ng et al. 1999) holds under stochastic PhysX. The efficiency comparison against the improvement reward is pending.

[Ng et al. 1999] [Grześ 2017] [Harutyunyan et al. 2015]

Computational Overhead – Self-Play Is Not Slower

Controlled measurement (same machine, same 528 envs, same cuRobo IK + physics):

PPO-PBRS (single-agent) vs the self-play variants run at $\sim 1.0\times$ per-iteration wall-clock (~ 0.022 it/s for both). The 2-agent machinery (Alice + Bob, GoalEncoder, ABC, historical pool) adds **no** measurable per-iteration overhead – the shared cuRobo IK + PhysX contact step dominates the cost.

Key result:

- Self-play is **not** slower per iteration – it simply **does not learn**
- The real cost of self-play is model/code complexity and added failure modes (two networks, non-stationary objective, ABC), *not* throughput
- So the $4.8\times$ performance gap is **not** a compute-budget artifact – both used the same compute



Limitations & Future Work

1. **Bob penalty collapses to 0%** – penalty on catastrophe phase ends creates a “fail-fast” equilibrium.
→ **Zero-sum game theory** (Letcher 2019) may stabilise symmetric self-play; test penalty only on non-catastrophe ends.
2. **Combined pos+rot gate is the universal bottleneck** – joint gate rarely fires under self-play (0–10% pos+rot).
→ **Relaxed success gate** or partial credit per objective to provide gradient toward both.
3. **Single-seed validation** – training SR variance across 5 chains (BestSR 0.081–0.110).
→ **Re-evaluate** checkpoints across 5 seeds for validation confidence intervals.
4. **Curriculum substitution is regime-dependent** – PBRS substitutes for curriculum at batch 7,920; curriculum helps at batch 960.
→ **Batch-size sweep** to map the substitution boundary and gradient-variance threshold.



university of
 groningen

Thank You – Questions?

Vlad Iftime s3426394

Faculty of Science and Engineering, Artificial Intelligence
University of Groningen

June 2026



Appendix Outline

- A. Push Mechanics – Limit Surface & Characteristic Length L
- B. Training curves for all PBRs models (Success Rate, Reward, Loss)
- C. PBRs parameter sensitivity (k_p , k_r , w_{pos} , w_{rot})
- D. Validation scene descriptions (30 test cases)
- E. cuRobo IK pipeline and workspace configuration
- F. Observation space specifications (all model variants)
- G. Per-test error bars – PPO-PBRs vs PPO-Curriculum head-to-head and all-model comparison (95% binomial CIs)
- **H. P82 Curriculum Trigger Bug** – why the original trigger never fired
- **I. BobPenalty Fail-Fast Equilibrium** – degeneracy mechanism
- **J. Confounded Ablation Disaggregation** – 7-axis variation table
- **K. Training Budget Comparison** – envs, iterations, push-macros
- **L. Validation Protocol** – scene set, trials, disclosed limitations



PBRS – The Shaping Theorem

Ng et al.:

$$\mathcal{R}'(s, a, s') = \mathcal{R}(s, a, s') + \underbrace{\gamma\Phi(s') - \Phi(s)}_{\text{shaping reward } F}$$

Policy invariance: $Q'(s, a) = Q(s, a) - \Phi(s)$ – optimal actions unchanged.

Our potential functions (PPO-PBRS / PPO-Curriculum):

$$\Phi(s) = w_{\text{pos}} e^{-k_p d_{\text{pos}}^2} + w_{\text{rot}} e^{-k_r c(s)}, \quad c(s) = \frac{1 - \cos(\theta^* - \theta)}{2}$$

$k_p=30, k_r=5, w_{\text{pos}}=w_{\text{rot}}=10.0$

Dense reward: $F(s, s') = \Phi(s') - \Phi(s)$ – difference only, no constant drain.

Episodic PBRS : $\gamma_{\text{shaping}}=1.0$ valid – $\sum F = \Phi(s_T) - \Phi(s_0)$ bounded.



PBRs – Why It's the Right Choice

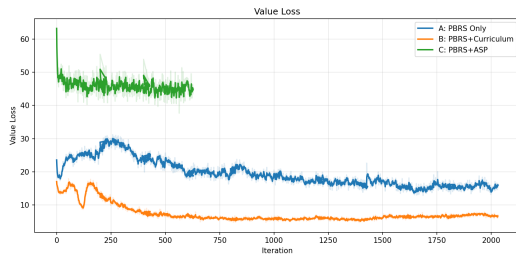
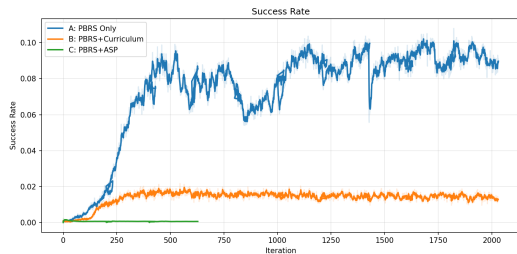
Properties of $F(s, s') = \Phi(s') - \Phi(s)$:

1. **Markovian** – depends only on s, s' , not history.
2. **Bounded** – exponential potentials prevent reward explosions from PhysX contact spikes.
3. **Zero-sum cycles** – detour pushes net to zero; agent free to explore without reward hacking.
4. **No constant drain** – $F=0$ when stationary; no penalty for hesitation or deliberation.

Key consequence: you can replace any ad-hoc reward with a potential-based one *without changing what the agent learns* – only *how fast* it learns. Reward design and policy optimisation are decoupled.



PPO-PBRS – Training Dynamics



Left: PPO-PBRS (blue) steadily improves training success rate over 3000 iterations. PPO-Curriculum (orange) follows a similar trajectory but converges slightly lower. Self-play variants remain near baseline throughout. **Right:** Stable value function – no explosion. Initial $V \approx 0.057$ with gain= 0.01.



PPO-Curriculum vs PPO-PBRS

Metric	PPO-PBRS	PPO-Curriculum	Gap
Validation SR	80.0%	76.7%	1.04×
PosErr (m)	0.032	0.023	B 28% lower
RotErr (rad)	0.568	0.663	A 14% lower
Pos-only SR	100%	100%	equal
Pos+rot SR	70%	65%	5pp

Result: Curriculum improves position precision (0.023 vs 0.032 m) from the pos-only phase, but slightly degrades rotation (0.663 vs 0.568 rad). Net effect: B is 3.3pp behind A. **Statistical note:** The 3.3pp A vs B gap is **not** significant ($p > 0.05$, two-tailed z-test; SE of difference = ± 10.6 pp across 30 independent scenes). Both A and B solve the task similarly well. **Takeaway:** A staged curriculum *can* work alongside PBRS, but does not beat the simpler no-curriculum baseline.



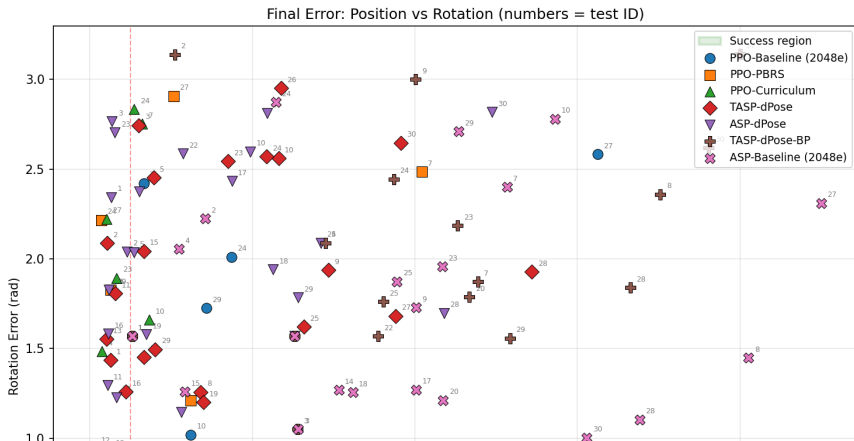
Self-Play – Validation Comparison

Model	Valid SR	PosErr	RotErr
PPO-PBRS (single-agent, ref.)	80.0%	0.032 m	0.568 rad
TASP-dPose (time-based)	16.7%	0.158 m	1.457 rad
ASP-dPose (outcome-based)	6.7%	0.143 m	1.612 rad

Key findings:

- Time-based self-play (TASP-dPose) is $2.5\times$ better than outcome-based (ASP-dPose), but still **$4.8\times$ below** single-agent PPO-PBRS
- No self-play variant achieves pos+rot SR above 10% – the combined gate remains the bottleneck
- Self-play complexity provides no benefit

Comparison Summary – Per-Scene Position × Rotation Error



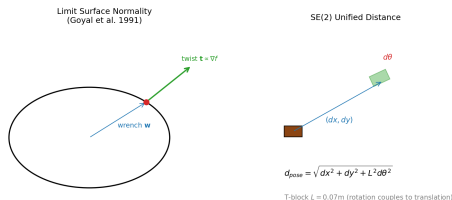
Appendix A – Push Mechanics: Limit Surface & L



Limit surface normality: $\mathbf{t} \propto \nabla f(\mathbf{w})$ – twist is normal to the surface, so every push couples translation *and* rotation.

Characteristic length L (radius of gyration):

- **T-block** $L = 0.07$ m – an off-centre push couples translation *and* rotation
- L is a physical constant, *not* a hyperparameter

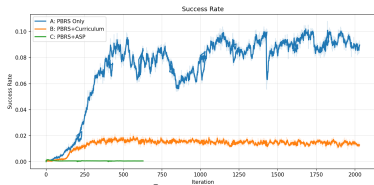


The T-block rotates under an off-centre push (coupled SE(2) motion)

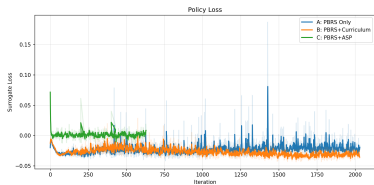
Appendix B – Training Curves (All PBRs Models)



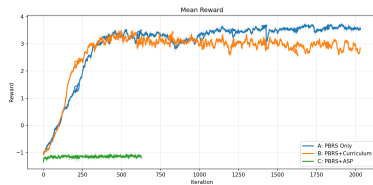
Success Rate



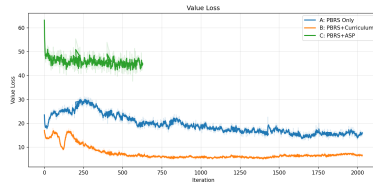
Policy Loss



Mean Reward



Value Loss

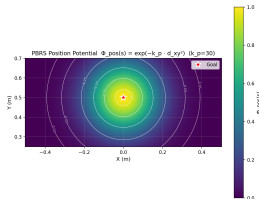


Appendix C – PBRs Theory Analysis



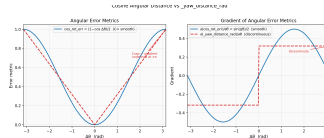
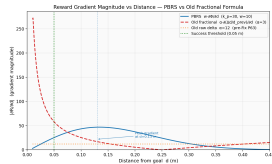
Potential Landscape

$$\Phi(s) = w \cdot \exp(-k \cdot d^2)$$



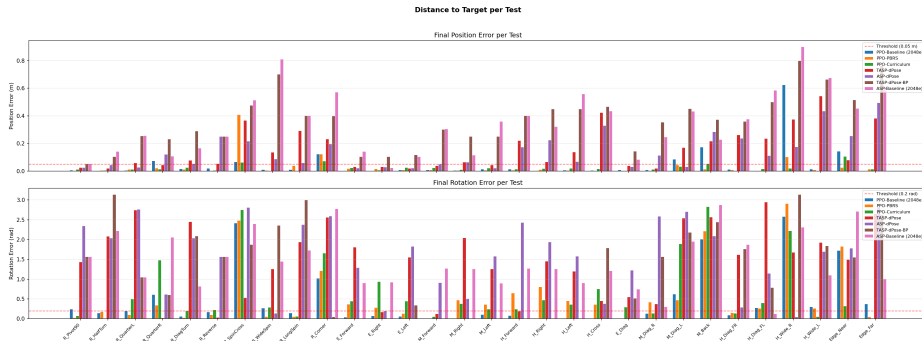
Gradient Comparison

PBRs vs Fractional Formula



Cosine Angular Distance $c(s) = (1 - \cos(\theta^* - \theta))/2$

Appendix G – Per-Test Error Bars (95% Binomial CIs)



Top: PPO-PBRS (blue) vs PPO-Curriculum (orange) head-to-head – per-scene trial-level SR from 20 trials. Overlapping CIs on most scenes.

Bottom: All 4 models – the self-play variants are near-zero on all but the easiest scenes.

Green labels: position-only tests. **Purple labels:** position+rotation tests.



Appendix H – P82 Curriculum Trigger Bug

The original PPO-Curriculum trigger never activated because it gated on an unreachable metric.

Broken trigger (original):

If $\text{mean}(\text{per-push PosErr}) < 0.08 \text{ m}$ for ALL 50 consecutive iterations $\Rightarrow$ activate Phase 2

Why unreachable:

- Per-push PosErr is measured after *every* push, including early ones far from a freshly randomized goal (0.1–0.45 m away)
- The iteration-mean has a structural floor well above 0.08 m – even the *best* model's validation PosErr is 0.202 m
- 50-iteration AND gate: one noisy iteration resets the streak

Consequence: $w_{\text{rot}} = 0.0$ permanently – rotation received zero reward signal. Model B (the pre-P82 version) was effectively position-only PPO.

Fix (P82): gate on *episodic position-only SR* ≥ 0.5 over a 20-iteration lookback. This is a “position is solved” signal, not a per-push mean. Curriculum now activates

Appendix I – BobPenalty Fail-Fast Equilibrium



Model I (TASP-dPose-BobPenalty) adds a Bob time penalty $R_B += -\gamma_{sp} \cdot t_B$ on *all* phase ends, including catastrophes.

Why it creates a degenerate equilibrium:

Trajectory	PBRS + Sparse	Penalty ($0.5 \times t_B$)	Total return
Fail on push 1	-10.5	-0.5	-11.0
Fail on push 10	-15.0	-5.0	-20.0
Succeed push 5	+10.0	-2.5	+7.5

Mechanism:

1. PPO's gradient pushes Bob toward higher returns – “fail on push 1” (-11.0) beats “try for 10 pushes and fail” (-20.0)
2. Since failing is easier than succeeding in contact-rich physics, the policy collapses to immediate failure
3. Alice's reward collapses simultaneously: $R_A = 0.5 \cdot \max(0, 1 - 5) = 0$ – she gets zero signal⁶³

Appendix J – Confounded Ablation Disaggregation

Self-play variants differ from single-agent PPO-PBRS on ≥ 7 axes simultaneously:

Variable	PPO-PBRS (80.0%)	Self-play (6.7–16.7%)
Agent count	1	2 (Alice + Bob)
Goal distribution	Fixed random	Adversarial, shifting
GoalEncoder	None	ϕ -MLP 8D latent
ABC buffer	None	Yes (ASP-dPose) / No (TASP-dPose)
Historical pool	None	Yes
Episode budget	15 pushes/agent	5 (Alice) + 10 (Bob)
PPO updates/iter	1	2 (Alice, then Bob)

Implication: the $4.8\times$ SR gap cannot be attributed to any single lever.

- TASP-dPose vs ASP-dPose isolates Alice’s reward type – TASP-dPose gains +10pp
- But this is a single 1-axis experiment within a 7-axis package change
- We present “self-play as implemented,” not “the curriculum mechanism in isolation”



Appendix K – Training Budget Comparison

All PBRs variants used identical budgets. The PPO-Baseline used more compute for a direct architectural comparison.

Model	Envs	Max Iter	Est. Push-Macros
PPO-Baseline (Fractional)	2,048	100,000	~3.07B
PPO-PBRs	528	3,000	~23.8M
PPO-Curriculum	528	3,000	~23.8M
ASP-dPose	528	3,000	~23.8M
TASP-dPose	528	3,000	~23.8M
TASP-dPose-BP	528	3,000	~23.8M

Caveats:

- PPO-Baseline at 2,048 envs received $\sim 129\times$ more push-macros than PBRs models
- Both converge to 80% validation SR – PBRs achieves this with $4\times$ fewer parallel envs and a fraction of the total push budget



Appendix L – Validation Protocol

All models evaluated under an identical protocol:

- **Scene set:** 30 held-out T-block scenes – 10 rotation-dominant (R_*), 10 position-only ($E_*/M_*/H_*$), 10 combined position+rotation
- **Trials:** 20 independent trials per scene (best-of-20); a scene is “solved” if *any* trial meets the gate
- **Success gate:** $\text{pos_err} < 0.05 \text{ m}$ **and** $\text{rot_err} < 0.2 \text{ rad}$ at trial termination
- **Push budget:** up to 30 pushes per trial (training uses 5–15)
- **Scene SR vs Trial SR:** scene SR counts scenes where $\geq 1/20$ trials pass; trial SR counts individual trial success rate

Disclosed limitations:

- **Single-seed validation:** each model evaluated once (one seed per checkpoint). Training SR variance exists across 5 chains (PPO-PBRS BestSR 0.081–0.110) but validation CIs are single-model.